Enhancing Win Probability Analysis in Counter-Strike through Position Embeddings with PureSkill.gg

Kamal Chahrour
School of Computer Science
Carleton University
Ottawa, ON, Canada
KamalChahrour@Cmail.Carleton.ca

ABSTRACT

Understanding player performance and projecting match results are critical components of the ever-changing esports scene that continue to interest researchers, fans and players. This paper explores the realm of competitive gaming, concentrating on arguably one of the greatest first-person shooters of our generation, Counter-Strike:Global Offensive (CS:GO for short). We start by giving a thorough overview of Counter-Strike, going over its core ideas, mechanics and tactics to give a better overview of what influences win probabilities and player performances such as crucial kills, exit frags, bomb defuses all the way to just surviving the round despite losing.

Next, we turn our focus to PureSkill.gg, an analytical framework that aims to evaluate player skill and forecast matches results in the Counter-Strike realm. Leveraging PureSkill.gg's current data and past research on win probability calculations to enhance current state-of-the-art win probability calculations using position embeddings. This allows the analysis of much more important data such as how a team plays a round using their positioning, which is much more essential than how individuals perform, as a team's harmony in a music symphony is more important than the skill of individual instruments; a masterpiece is created by the seamless blending of tones and rhythms. Likewise, in the world of Counter-Strike, knowing how a team moves between areas to create strategic gaps or false truths to the enemy team is what sets apart teams with higher skill, which is what we will be analyzing.

CCS CONCEPTS

• **Applied computing** → **Computer games;** • **Information systems** → Data mining.

KEYWORDS

Counter-Strike, win probability, position embeddings, first person shooter, word embeddings, machine learning, accuracy

ACM Reference format:

Kamal Chahrour. 2023 Enhancing Win Probability Analysis in Counter-Strike through Position Embeddings with PureSkill.gg. In *Proceedings of ACM Interim Template.* https://www.acm.org/publications/proceedings-template

1 INTRODUCTION

Counter-Strike is a first-person shooter which consists of two teams, namely the Terrorist side opposing the Counter-Terrorists, we'll call them Ts and CTs as everybody does in the Counter-Strike community. Each team fights to be the first to reach 16 round wins through 2 halves of gameplay, consisting of 15 rounds per side for each team. The most influential aspect in terms of winning a Counter-Strike is without a doubt teamplay followed by individuals' mechanics. However, what really wins games is when players find gaps in the opponent's defense and use their superior positioning to gain advantages that would otherwise be hard fought and disadvantageous if they were to just take the fight head on. Not to confuse with the newest edition of the franchise recently released, Counter-Strike 2 which has new half dimensions being 12 rounds each with a first to 13 round wins being the winner, we will be using data from CS:GO which contains 15 round half and requires 16 round wins to win the game.

PureSkill.gg provides a thorough method for improving player performance and forecasting Counter-Strike match results. This intuitive interface integrates with players' gaming data from Matchmaking and *FACEIT*, a private platform for the most competitive players. The AI coach on the platform evaluates a variety of gameplay of aspects through the use of complex algorithms. These aspects include individual mistake detection, economy management analysis, round impact, grenade usage, crosshair placement, teamwork, shooting accuracy, movement proficiency and decision-making skills. In contrast to conventional benchmarks, this AI coach customizes its comments based on the player's playstyle, performance, influence on victory and intricacy of fault rounds. With the goal of revolutionizing esports analytics, PureSkill.gg offers players looking to advance their abilities and climb the ladder individualized insights and practical tips. PureSkill.gg has an open data policy and links to data documentation can be found here https://pureskill.gg/data. This data source has hundreds of thousands of matches from which we can get data from. Each match is broken into 20+ data tables which can be linked together through timestamps and player IDs. In particular, one table is a player vector channel that

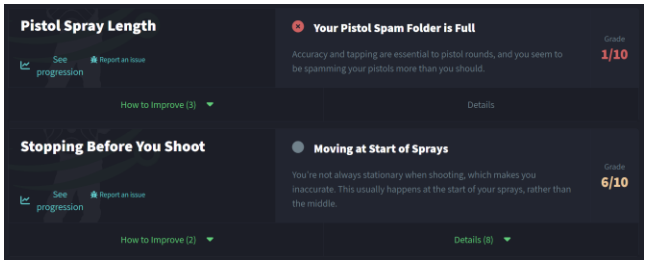


Figure 1: PureSkill.gg showing which areas a player needs to improve on from their most recent match.

provides positioning information extracted at 64 or 128 times per second.

As seen in Figure 1, the AI coach of PureSkill.gg provides a detailed analysis on where a player could have messed up and improved with exact links to what can be done to improve by clicking on the how to improve button. As you can see, the coach focuses on mechanics in this part of the coach specifically calling out this players Pistol Spraying which is heavily advised against as aiming for heads and shooting slowly in pistol rounds is key.

One part of the AI coach that is lacking is the factor of not considering positions of players when calculating win probabilities in the match viewer. Looking at the paper on this very subject on PureSkill.gg's Win Probability Model [1] where they show that a side's HP, armor, equipment, and players alive contributed the most to their predictions. Xenopoloulos et al. [2] have used professional Counter-Strike: Global-Offensive replays to create a game-level win probability model based on factors such as available utility, current weapons, number of players alive, health and armor of each player and the phase of the round with how much time is left. The main drawback of all these works being that positions are not factored in at all, possibly one of the most important factors of determining who is going to win the current round. A team that is coordinated beyond incompetence will very rarely lose to a team that is less coordinated but similarly rated in terms of mechanics and skills, furthermore, proving that positions of players in comparison to their teammates and the enemies do matter in calculating win probability.

Win probability is used to measure a player's impact on a round. Essentially, by subtracting the win probability before an action from the probability after an action, you can measure the impact of an action. This work will add positioning as a feature to current machine learning models. Positioning is important because one players positioning can alter the entire course of a round, and perhaps the game if its just that impactful. A good example of this could be as simple as when 4 players decide to attack a site and leave a lurker on the other side of the map, ready to pinch when the other 4 cause a distraction and potentially allowing the 4 other players to wrap back around to where they left the 5th player on the other extremity to a now free site due to rotations that occurred from all the yelling on the enemy team calling for backup on the original site. This shows just how impactful positioning can be in a game and how important it is to not neglect this part of the game in pursuit of improvement of ones skill. This addition to the tool of PureSkill.gg AI coach will help better understand and show players just how impactful a factor positioning is in games. All code can be publicly accessed at https://github.com/kamalchah/CounterStrikeWinProb.

1.1 My Experience in Counter-Strike

The very reason I am taking on this challenge is because I have used this very tool in the past to improve upon my skill when I was initially getting into competitive play. A very good indicator of this AI coach being very useful is my use case, I went from being a beginner player in 2020 being level 3 on the *FACEIT* platform (10 total levels, Level 10 is attained once one reaches 2000+ Elo, after this it scales as Elo with everybody being level 10, one can descend and ascend levels) to present, being level 10 with 2200 Elo, a Elo-level less than 0.01% of the population of the *FACEIT* community have ever reached. I also play in a competitive league on *FACEIT*, called ESEA which used to have its separate platform but recently had a merger with *FACEIT* making it essentially the same entity, also combining their websites. The league I play in is ESEA Main, the 3rd tier league having the 2nd tier league called ESEA Advanced, and the 1st tier league called ESL Challenger League, where the best of North America competes for a cash prize every Season, currently on Season 47 and entering Season 48 in January of 2024.

My hope is to use my in depth understanding of the game and my outlook on what provides a team winning outcomes and overall win probability increasing scenarios to implement into the AI coach of PureSkill.gg using player position embeddings as recently, one of my main focus' on my game has been understanding how to play the game as if it were a game of chess, moving each player tactically with reason and intent and never for no reason. A good example of this would be when playing on the Counter-Terrorist (CT) side, your job is to stop the Terrorists (T) from either planting the bomb or eliminating the entire enemy team. Counter-Terrorists will often setup in ways to setup crossfires, baiting in the T-side to take fights from one side of a bombsite or general area, allowing a outside player to take contact against players who are focused on another player. The problem with this scenario is as you scale skill levels, the more this exact scenario changes. Very often, lower skilled players will leave gaps in defense. Position embedding can provide an estimate and show just how much positioning can be improved upon compared to other skill levels. At the very bottom level this will work flawlessly, climb to the pro level and suddenly only 1 player will be distracted by the crossfire with the other players being hyper aware of a crossfire being a very strong possibility and setup by the CT side in an effort to counter a strong T side execute or push. This is just a small example of a scenario showing just how in-depth skill can scale in this game. Position embeddings will allow us to reach better conclusions on just how impactful it is and how

the average player can improve their positioning when compared to the pros of which our data set will contain professional games.

2 RELATED WORKS

The problem of analyzing win probabilities in Counter-Strike is not as simple as it seems. With so many different factors to consider ranging from the number of players on either team alive or dead, the amount of equipment each player has, the guns each players have or even the amount of health each player has left. This particular problem has attracted the interest of various researchers or just curious Counter-Strike enthusiasts [3,4]. Studies performed on this problem have mainly focused on the previously mentioned aspects of the game along with some extra ones but don't make any mention of using position embeddings at all.

Some studies showed that the starting point of a player in a map can correlate to interesting metrics however, they don't specifically use the positioning of the players during the round but instead only at the beginning of the round to see how impactful their spawn was on the map, as players in Counter-Strike have random spawns in a given area called either T or CT-spawn. We can imagine a small range of spawn points in a given teams spawn, with some maps having up to 20 different spawn points in the spawning area of players which can lead to fine tuned strategies that make use of proper timings of spawns which can be considered advantageous to the current round. However, this is not what we are interested in. The particular point of this paper is to use position embeddings to help improve the prediction of win probability in Counter-Strike.

To the best of our knowledge, this is the first study that carries out a fairly comprehensive analysis and various experiments to present the use of position embeddings through means of word embeddings to help give more context to our win probability calculator.

3 EXPERIMENTS

In this section, we conduct an in-depth experimental explanation of how we setup our experiments and training sets before the actual benchmark was run. We show the setup of our XGBoost input parameters along with word2vec.

3.1 Data used for training and validation.

To considerably narrow our results and training time, we decided to focus our input data to be from only 1 map from Counter-Strike being Mirage (code name de_mirage). The reasoning for this is the extensive time it takes to read in the folders of which contain over 30 parquet files per match can become quite extensive and can start to take a while to parse before completely bricking the machine at hand. We will parse 200 different games for this experiment, showing valuable data insights as well as some remarks on the data.

3.2 Machine specifications

All parsing, analyzing and running of any algorithms were done on my personal machine running on a 12th Gen Intel® Core™ i5-12600KF, AMD Radeon RX 6700 XT and 47.8 GB of 3200 MHz memory.

3.3 Training Dataset

Before we begin talking about the training dataset and how we obtained it, we must first take a look at how all the demo's (recorded game files) are parsed and structured. These folders contain 33 parquet files which are read into our python file using data frames from Pandas, which are then stored in our local memory code. This is perhaps the most tedious part, taking the longest and needs to be re-run anytime we make a modification to the way the data is parsed of used in our training dataset. Most of our calculations occur around 4 files, namely ***player_vector***, ***player_status***, ***header*** and ***round_end***. These parquet files contain all instances of player movements, player status and general match information such as map name and tick rate in the ***header*** parquet file and some more important attributes and columns in ***round_end*** which are used in our importance graphs later.

For our training dataset without position embeddings, we extract the number of T players alive, the number of CT players alive, the equipment value on both sides and which team won the round for 1 random tick per round per game. This ensures that we have an independent dataset on which we can generalize our XGBoost classifier on. The reason for not taking more than 1 tick per round is to make sure that our dataset is independent and not biased as having more than 1 tick per round in our dataset could skew the results.

3.4 Training Dataset with word embeddings

For our training dataset with position embeddings, we employ the use of word embeddings and word2vec to represent our textual data and essentially capture semantic relationships between words. We will be using words to portray the positioning of players in this experiment. The construction of these words can be simplified to the extraction of the *x_pos* and *y_pos* columns by representing the positions of players in the current tick using conditions to make sure this is the case, followed by dividing these values by 256 and concatenating them per player to obtain a string like representation of all alive players in the current tick iteration of the round at hand.

Below is an example and for simplicity's sake, we will assume only 1 player on both sides is alive to make for a shorter example. The algorithm starts by checking for alive CT players, followed by filling in the cells associated with this players coordinates divided by 256 with rounding and filling the rest with separators, 'a' in this case and filling in the x,y values for the dead players as 0. The string construction will start by adding the next T-side players values and filling in the rest of the cells for dead players as 0. A separator is in the middle represented as _ to show the left side of this separator as the CT-side coordinates, and the inverse as the T-side coordinates as a word embedding.

Example 1: T-side Players: Joe @ 1472,1928 (x,y)

CT-side Players: Steve @ 1322,1758 (x,y)

1 CT player alive thus:

Round(1322/256)= 5

Round(1758 /256)= 7

String construction (CT): 5,7a0,0a0,0a0,0a0,0a0,0_

1 T-side player alive thus:

Round(1472/256)= 6

Round(1928 /256)= 8

String construction (T): 6,8a0,0a0,0a0,0a0,0a0,0

Complete string representation for the current tick, of the current round: **5,7a0,0a0,0a0,0a0,0a0,0_6,8a0,0a0,0a0,0a0,0a0,0**

Having this entire process streamlined as an algorithm which is called upon when we are constructing our training dataset for word2vec which is separate to our XGBoost classifier.

3.5 Training word2vec

To incorporate word embeddings into our prediction model, XGBoost, it is important to train word2vec on sentences that adequately represent the spatial information of players positions in the game. Word2vec is a well-known technique for learning distributed representations of words, capturing semantic relationships between them. In our case, each 'word' in a sentence represents a certain tick's entire positioning information, meaning all 10 players in the game. To train word2vec, we need to parse all our demos' iterating over the start of the round until the end, skipping 3 seconds in between words. We end up with a sentence ranging anywhere from 10 words all the way to 60 depending on how long the round lasts and how many valid ticks we are able to extract. Initializing word2vec with a large array of sentences containing the word embedding representation of the players' positions over all the demos, skipping 3 seconds in between words will yield us a large and diverse enough dataset for us to generalize word2vec on. The primary goal of training word2vec in this context is to be able to query it with another word generated from the player positions in the current tick and be able to extract vectors that represent the spatial context of players during a specific game moment. Using this 3-second skip approach, we ensure that the model learns not only the immediate spatial relationships but also the relationship between the transforming positions throughout the round making it considers the positions further apart in time while enhancing its ability to understand strategic movement and gameplay dynamics. In essence, we can transform positional information into dense vectors which can then be input into our XGBoost model as another attribute of the current round.

4 RESULTS

In this section, we conduct an in-depth evaluation of the 200 games parsed. We present the results from our benchmark comparing the results of our models with and without position embeddings using word embeddings. We show feature importance and how much effect each parameter had on the output.

4.1 Analysis of results

Our goal in this paper is to improve the accuracy of the given model from [1] and [2] which we seem to have achieved. Through the confusion matrix, we can measure the performance of our algorithms using the accuracy of the True positives and True negatives over all instances of unique results.

Our results indicate the accuracy of the XGBoost classifier without word embeddings has an average of 0.6477 over 3 individual runs using randomized data from our 200 games, whereas the accuracy of the XGBoost classifier with word embeddings using the player positions has an accuracy of 0.67486 showing that we retrieve a greater accuracy of our model using the actual position embeddings of players. We didn't quite reach this result initially as we had to tune some parameters belonging to either XGBoost or word2vec such as XGBoost's max depth which seems to be optimal at a value of 8, whereas word2vec's parameters have a window size of 5 and a vector size of 8 which is shown as the 8 vectors in the figure 2.

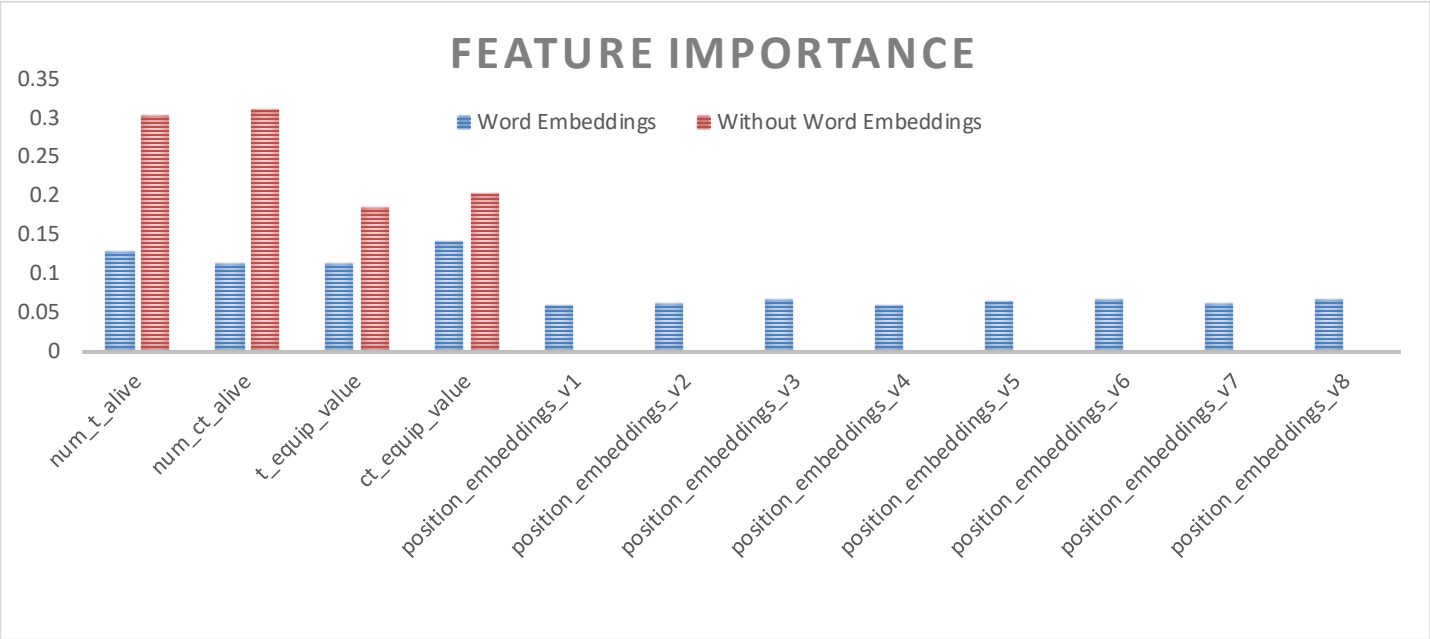
Although we managed to reach a higher accuracy using position/word embeddings, the F1 score was on average higher for the algorithm without word embeddings, showing a value of 0.7 on average and an average of 0.65 with position/word embeddings.

On top of the increased accuracy using the word embeddings of positions, we can see a average value of around 0.063 on vector importance denoted as position_embeddings_v1 through v8.

4.2 Remarks

We also notice a ratio of *num_t_alive* and *num_ct_alive* slightly shifted in the word embeddings section. We can see that instead of these values being nearly equal with *num_ct_alive* slightly greater, we can now see a greater importance shifting towards the *num_t_alive* attribute showing that the importance of the number of T players alive is more important in the context of player positions which, as a Counter-Strike enthusiast, I can agree on this finding being one in the right direction as T player lives are often valued more in the presence of position context as the impact that can be had on the round by 1 T player alone is arguably greater than that of it's counterpart, 1 CT player being able to have as much impact on the round. We can also infer that the position vectors values have taken some of the importance away from the number of alive players on each side.

Along with this finding, we can also make another remark. The importance of *t_equip_value* and *ct_equip_value* can be seen as having the same relationship with and without word embeddings however, through this we can also remark that through position embeddings, the algorithm still weighs the



importance of CT's using their equipment over the T's using their equipment which is an objectively correct statement to make in the

Figure 2: Feature importance for both models

presence of context as a CT's utility will typically be more important as it is the CT's utility which can keep away the T's from the bombsite (objective).

Lastly, we observe the 8 vectors with similar importance between all of them as expected as it is one word that is being represented here through the word2vec model's query when we are taking 1 random tick per round to create a word and enter that word's vector into the model.

5 POTENTIAL IMPROVEMENTS

Although we managed to improve upon the existing framework by [1] and [2], we did not improve by much and the F1 score turned out to be higher for the algorithm without position/word embeddings. Some potential improvements could range from adding more data attributes such as guns used, current health at current tick per player, adding a live demo viewer to be able to see live win probability odds for all ticks of a round to be able to see how accurate it is in real time. This is just a hint of the improvements we could make, and we haven't begun to discuss the hyperparameters of XGBoost of word2vec yet. Some other potential improvements could lie in our calculations such as the amount we divide the coordinates by to represent as words could be changed from 256 to say, 64 to be more descriptive and accurate in certain positions as we could be generalizing too much by dividing too high. Another improvement could lie within the amount of words gathered per round, generating a word every 3 seconds per round for the word2vec training could prove to not be enough or too much. We could also try to use neural networks instead of XGBoost to try and leverage other machine learning algorithms or libraries to try and get a more accurate representation of how position/word embeddings influence results.

A big potential improvement could be in the map used, we chose to use a map, mirage, which is historically known for being very CT-sided as the T-players often engage in unfavorable duels at not so good timings meaning the very basis of this entire algorithm could be flawed, however all maps contain certain biases and will yield their own results depending on the maps identity.

5 CONCLUSION

In this paper, we described the remarks learnt from our benchmark which was run on the framework proposed by [1] and [2] which we upgraded by adding position embeddings through means of word embeddings in a XGBoost model. Our experience shows that certain patterns were able to be pulled from the feature importance graph as well as the overall accuracy of the model with and without position embeddings. I am confident through our findings that the Counter-Strike data science community can make groundbreaking findings in the context of using position embeddings to calculate win probability. All code can be publicly accessed at https://github.com/kamalchah/CounterStrikeWinProb.

REFERENCES

[1] Peter Xenopoulos, William Robert Freeman, and Claudio Silva. 2022. Analyzing the Differences between Professional and Amateur Esports through Win Probability. In Proceedings of the ACM Web Conference 2022 (WWW '22), April 25–29, 2022, Virtual Event, Lyon, France. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3485447.3512277

[2] Peter Xenopoulos, Harish Doraiswamy, and Cláudio T. Silva. 2020. Valuing Player Actions in Counter-Strike: Global Offensive. In IEEE International Conference on Big Data, Big Data 2020, Atlanta, GA, USA, December 10-13, 2020, Xintao Wu, Chris Jermaine, Li Xiong, Xiaohua Hu, Olivera Kotevska, Siyuan Lu, Weija Xu, Srinivas Aluru, Chengxiang Zhai, Eyhab Al-Masri, Zhiyuan Chen, and Jeff Saltz (Eds.). IEEE, 1283–1292. https://doi.org/10.1109/BigData50022.2020.9378154

[3] Rubin, Allen. "Predicting Round and Game Winners in CSGO." OSF Preprints, 13 Jan. 2022. Web. https://doi.org/10.31219/osf.io/u9j5g

[4] Švec, O. (2022). Predicting Counter-Strike Game Outcomes with Machine Learning. https://dspace.cvut.cz/bitstream/handle/10467/99181/F3-BP-2022-Svec-Ondrej-

predicting_csgo_outcomes_with_machine_learning.pdf?sequence=-1&isAllowed=y

[5] Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, pages 785–794. https://dl.acm.org/doi/10.1145/2939672.2939785